A Tour Of Sage

Release 10.10

The Sage Development Team

Oct 28, 2026

CONTENTS

Note

You can try the examples below in your browser right away, with nothing to install: just type them into the Sage Cell Server at https://sagecell.sagemath.org.

This is a short tour of Sage as a calculator.

The Sage command line has a prompt "`sage:`". To experiment with the following examples, you only enter the part after the prompt.

```
sage: 3 + 5
8
```

```
>>> from sage.all import *
>>> Integer(3) + Integer(5)
8
```

If you use Sage on the Jupyter notebook, then likewise put everything after the prompt in an input cell, and press `Shift-Enter` to get the corresponding output.

The caret symbol means "raise to a power".

```
sage: 57.1^100
4.60904368661396e175
```

```
>>> from sage.all import *
>>> RealNumber('57.1')**Integer(100)
4.60904368661396e175
```

We compute the inverse of a 2×2 matrix in Sage.

```
sage: matrix([[1, 2], [3, 4]])^(-1)
[  -2    1]
[ 3/2 -1/2]
```

```
>>> from sage.all import *
>>> matrix([[Integer(1), Integer(2)], [Integer(3), Integer(4)]])**(-Integer(1))
[  -2    1]
[ 3/2 -1/2]
```

Here we integrate a simple function.

```
sage: x = var('x')   # create a symbolic variable
sage: integrate(sqrt(x) * sqrt(1 + x), x)
1/4*((x + 1)^(3/2)/x^(3/2) + sqrt(x + 1)/sqrt(x))/((x + 1)^2/x^2 - 2*(x + 1)/x + 1)
- 1/8*log(sqrt(x + 1)/sqrt(x) + 1) + 1/8*log(sqrt(x + 1)/sqrt(x) - 1)
```

```
>>> from sage.all import *
>>> x = var('x')   # create a symbolic variable
>>> integrate(sqrt(x) * sqrt(Integer(1) + x), x)
1/4*((x + 1)^(3/2)/x^(3/2) + sqrt(x + 1)/sqrt(x))/((x + 1)^2/x^2 - 2*(x + 1)/x + 1)
- 1/8*log(sqrt(x + 1)/sqrt(x) + 1) + 1/8*log(sqrt(x + 1)/sqrt(x) - 1)
```

This asks Sage to solve a quadratic equation. The symbol == represents equality in Sage.

```
sage: a = var('a')
sage: S = solve(x^2 + x == a, x); S
[x == -1/2*sqrt(4*a + 1) - 1/2, x == 1/2*sqrt(4*a + 1) - 1/2]
```

```
>>> from sage.all import *
>>> a = var('a')
>>> S = solve(x**Integer(2) + x == a, x); S
[x == -1/2*sqrt(4*a + 1) - 1/2, x == 1/2*sqrt(4*a + 1) - 1/2]
```

The result is a list of equalities.

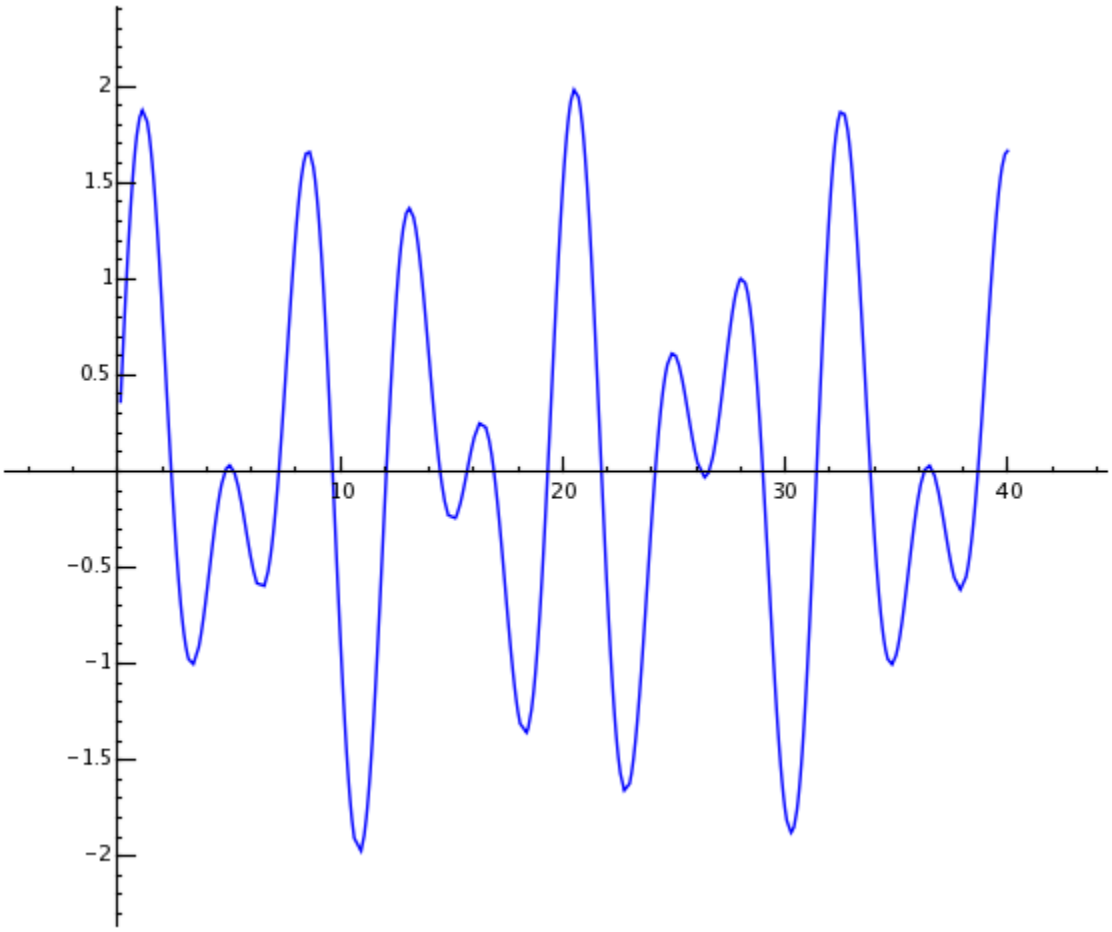
```
sage: S[0].rhs()  # right hand side of the equation
-1/2*sqrt(4*a + 1) - 1/2
```

```
>>> from sage.all import *
>>> S[Integer(0)].rhs()  # right hand side of the equation
-1/2*sqrt(4*a + 1) - 1/2
```

Sage can plot various useful functions, of course. (In the Jupyter notebook or the Sage Cell Server the figure appears inline; at the `sage` terminal prompt, `show` opens it with an external viewer.)

```
sage: show(plot(sin(x) + sin(1.6*x), 0, 40))
```

```
>>> from sage.all import *
>>> show(plot(sin(x) + sin(RealNumber('1.6')*x), Integer(0), Integer(40)))
```



Sage is a very powerful calculator. To experience it, first we create a 500×500 matrix of random numbers.

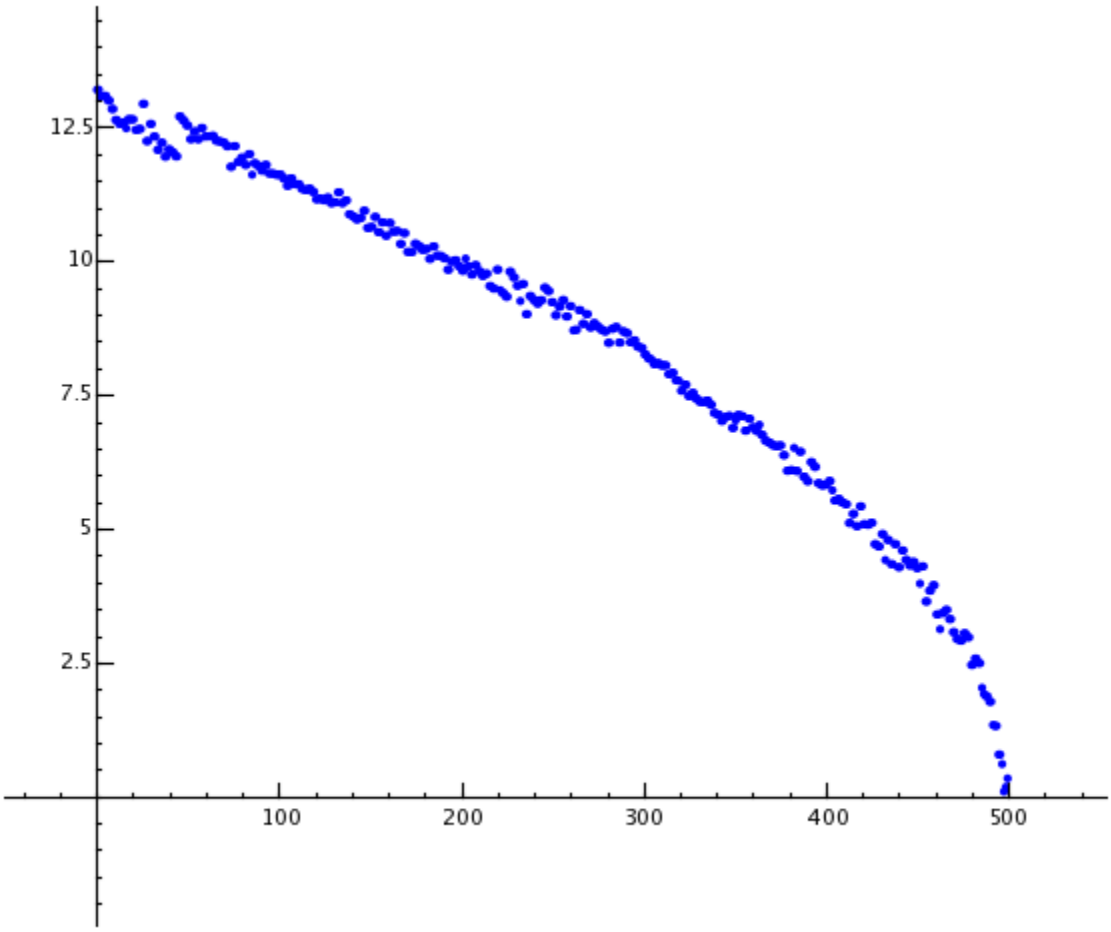
```
sage: m = random_matrix(RDF, 500)
```

```
>>> from sage.all import *
>>> m = random_matrix(RDF, Integer(500))
```

It takes Sage a second to compute the eigenvalues of the matrix and plot them.

```
sage: e = m.eigenvalues()  # about 1 second
sage: w = [(i, abs(e[i])) for i in range(len(e))]
sage: show(points(w))
```

```
>>> from sage.all import *
>>> e = m.eigenvalues()  # about 1 second
>>> w = [(i, abs(e[i])) for i in range(len(e))]
>>> show(points(w))
```



Sage can handle very large numbers, even numbers with millions or billions of digits.

```
sage: factorial(100)
933262154439441526816992388562667004907159682643816214685929638952175999932299156089414639761565182862536979208272237582511852109168640000000000000000000000000
```

```
>>> from sage.all import *
>>> factorial(Integer(100))
933262154439441526816992388562667004907159682643816214685929638952175999932299156089414639761565182862536979208272237582511852109168640000000000000000000000000
```

```
sage: n = factorial(1000000)  # about 1 second
sage: len(n.digits())
5565709
```

```
>>> from sage.all import *
>>> n = factorial(Integer(1000000))  # about 1 second
>>> len(n.digits())
5565709
```

This computes at least 100 digits of π.

```
sage: N(pi, digits=100)
3.14159265358979323846264338327950288419716939937510582097494459230781640628620899862
↪8034825342117068
```

```
>>> from sage.all import *
>>> N(pi, digits=Integer(100))
3.14159265358979323846264338327950288419716939937510582097494459230781640628620899862
↪8034825342117068
```

This asks Sage to factor a polynomial in two variables.

```
sage: R.<x,y> = QQ[]
sage: F = factor(x^99 + y^99)
sage: F
(x + y) * (x^2 - x*y + y^2) * (x^6 - x^3*y^3 + y^6) *
(x^10 - x^9*y + x^8*y^2 - x^7*y^3 + x^6*y^4 - x^5*y^5 +
 x^4*y^6 - x^3*y^7 + x^2*y^8 - x*y^9 + y^10) *
(x^20 + x^19*y - x^17*y^3 - x^16*y^4 + x^14*y^6 + x^13*y^7 -
 x^11*y^9 - x^10*y^10 - x^9*y^11 + x^7*y^13 + x^6*y^14 -
 x^4*y^16 - x^3*y^17 + x*y^19 + y^20) * (x^60 + x^57*y^3 -
 x^51*y^9 - x^48*y^12 + x^42*y^18 + x^39*y^21 - x^33*y^27 -
 x^30*y^30 - x^27*y^33 + x^21*y^39 + x^18*y^42 - x^12*y^48 -
 x^9*y^51 + x^3*y^57 + y^60)
sage: F.expand()
x^99 + y^99
```

```
>>> from sage.all import *
>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2)
>>> F = factor(x**Integer(99) + y**Integer(99))
>>> F
(x + y) * (x^2 - x*y + y^2) * (x^6 - x^3*y^3 + y^6) *
(x^10 - x^9*y + x^8*y^2 - x^7*y^3 + x^6*y^4 - x^5*y^5 +
 x^4*y^6 - x^3*y^7 + x^2*y^8 - x*y^9 + y^10) *
(x^20 + x^19*y - x^17*y^3 - x^16*y^4 + x^14*y^6 + x^13*y^7 -
 x^11*y^9 - x^10*y^10 - x^9*y^11 + x^7*y^13 + x^6*y^14 -
 x^4*y^16 - x^3*y^17 + x*y^19 + y^20) * (x^60 + x^57*y^3 -
 x^51*y^9 - x^48*y^12 + x^42*y^18 + x^39*y^21 - x^33*y^27 -
 x^30*y^30 - x^27*y^33 + x^21*y^39 + x^18*y^42 - x^12*y^48 -
 x^9*y^51 + x^3*y^57 + y^60)
>>> F.expand()
x^99 + y^99
```

Sage takes less than 1 second to compute the numbers of ways to partition one hundred million as a sum of positive integers.

```
sage: z = Partitions(10^8).cardinality()  # about .1 second
sage: z
176051704594624914136037389467913520400 9...
```

```
>>> from sage.all import *
>>> z = Partitions(Integer(10)**Integer(8)).cardinality()  # about .1 second
>>> z
176051704594624914136037389467913520400 9...
```

Sage is the world's most advanced open source math software.